Replication Study of Fairness-Aware Group Recommendation with Pareto Efficiency

Brandon Hawfitch
Anna Hoitomt
CSCI 5123 Recommender Systems
University of Minnesota

Spring 2025

Abstract

This report details a replication study of the "Fairness-Aware Group Recommendation with Pareto-Efficiency" paper by Xiao *et al.* (2017). The original work tackles the challenge of maximising each group member's satisfaction while reducing intra-group unfairness. Our study first reproduces the paper's headline results and then broadens the empirical scope to utility/fairness variants that were mentioned but not tabled. We confirm the original trends, highlight nuances that emerge under alternative parameter settings, and discuss their implications for fairness-aware recommender design.

1 Introduction

Group recommendation systems aim to present an item list that pleases a collection of users with possibly divergent tastes. Xiao *et al.* (2017) introduced a multi-objective formulation that simultaneously maximises *social welfare* (overall utility) and *fairness* (equity of utility distribution) through a Pareto-efficient search. This replication revisits their core claims and explores unreported combinations of fairness metrics, utility semantics and hyper-parameters.

2 Original Study Overview

2.1 Motivation and Objectives

Group-recommendation differs from one-to-one recommendation in that a *single* list must satisfy multiple—often heterogeneous—users. Earlier work either (i) treated persistent groups as a "virtual user", or (ii) simply aggregated individual scores with heuristics such as *average* or *least-misery*. Xiao *et al.* (2017) argue that these approaches overlook the trade-off between *social welfare* (overall utility) and *fairness* (equity of utility). They therefore cast Top-K group recommendation as a two-objective optimization problem that explicitly seeks Pareto-efficient solutions.

2.2 Utility and Fairness Modeling

Let G be a group of $|G|$ users and $L = \{i_1, \ldots, i_K\}$ a length-K recommendation list. Relevance scores $r_{u,i}$ are obtained from an underlying personalized model.

Individual utility. Xiao *et al.* consider two semantics:

$$U_{\text{Ave}}(u, L) = \frac{1}{K} \sum_{i \in L} r_{u,i}$$

$$U_{\text{Prop}}(u, L) = \frac{1}{K} \sum_{i \in L} \frac{r_{u,i}}{\max_{j \in I(u,K)} r_{u,j}},$$

where $I(u, K)$ is user u's Top-K personal items

Social welfare. $SW(G, L) = \sum_{u \in G} U(u, L)$.

Fairness. Four well-established metrics are adopted:

$$F_{\mathrm{LM}}(G, L) = \min_{u \in G} U(u, L),$$
$$F_{\mathrm{Var}}(G, L) = -\operatorname{Var}_{u \in G} U(u, L),$$
$$F_{\mathrm{Jain}}(G, L) = \frac{\left(\sum_u U(u, L)\right)^2}{|G| \sum_u U(u, L)^2},$$
$$F_{\mathrm{MM}}(G, L) = \frac{\min_u U(u, L)}{\max_u U(u, L)}.$$

LM and MM protect the least-satisfied member; VAR and Jain encourage tight utility dispersion.

2.3 Pareto-Efficient Formulation

The joint objective is scalarized as

$$\max_{L:\,|L|=K} (1 - \lambda)\, SW(G, L) + \lambda\, F(G, L), \qquad (1)$$

with trade-off weight $\lambda \in [0, 1]$. Maximizing LM or MM is NP-hard even for fixed K. Consequently, the authors design two solution families:

- **Greedy-F**: a forward-selection heuristic that adds the item yielding the largest gain at each step;
- **IP-Relax**: an integer program whose $\{0, 1\}$ decision variables are relaxed to $[0, 1]$, solved as a convex program and then rounded.

2.4 Algorithms and Baselines

The study evaluates six methods:

- **Greedy-LM** and **Greedy-Var** (proposed);
- **LM-Ranking** and **Ave-Ranking**: rank items by, respectively, least-misery or average score;
- **SPGreedy**: sequentially pick items that maximize group *satisfaction probability*;
- **EFGreedy**: greedily maximizes expected F-measure.

2.5 Experimental Design

Datasets. MovieLens-1M and MoviePilot.
Group construction. Three 8-user group types are sampled via cosine similarity: random (RG), similar (SG) and diverse (DG).
Relevance semantics. Binary ($r \in \{0, 1\}$) and Borda-count ($r \propto K - \text{rank}$).
Evaluation. Precision@K, Recall@K, F1@K, DCG@K and nDCG@K for $K \in \{3, 5, 10\}$.
Hyper-parameters. $\lambda \in [0, 1]$; matrix-factorization with BPR fills missing ratings.

2.6 Headline Results

On both datasets Greedy-LM and Greedy-Var outperform all baselines on every accuracy metric. Introducing fairness does *not* harm accuracy; instead, it raises mean F1 by up to 14 %. The best trade-off arises at $\lambda \approx 0.8 - 0.9$, and "similar" groups are easier to satisfy than "diverse" ones.

3 Scope of Enhancements

3.1 Goals and Constraints

Our initial intention was to conduct the replication of all parameter permutations indicated in the original paper, as well as design and implement some potential alternative parameter metrics and corresponding functions. It was theorized that we could develop alternative metrics for utility, fairness, and social welfare that could potentially enhance performance. The search algorithms used in the original paper were limited, and we were hopeful that implementing more intelligent search algorithms would be an effective solution to group recommendation.

However, it was made apparent that the runtime requirements of these algorithms and the allotted completion time of the assignment were in conflict with one another. The Greedy-F algorithm operates by an exhaustive Pareto search, and must examine every-K subsets of the catalog, where K is the number of recommended items. At each of the K steps, it scans the entire candidate pool and recomputes utility for every user. With each metric function taking

Table 1: Parameter dimensions that **Xiao et al. (2017)** define in the text but do *not* report experimentally.

Dimension	**Defined in paper**	**Reported On**	**Unreported**
Individual–utility U	*Average*, *Proportional*	*Proportional* only	*Average* with any F, K, λ
Fairness objective F	LM, VAR, Jain, Min-Max	LM, VAR	Jain / Min-Max for every K, λ, U
Group size $\|G\|$	2, 4, 6, **8**	**8** only	Metrics for $\|G\| \in \{2, 4, 6\}$
List length K	3, 5, **10**, **20**	**10** and **20**	Results for $K = 3$ and $K = 5$
Trade-off weight λ	Continuous $[0, 1]$	0.0-1.0	N/A, continuous value
Relevance $r_{u,i}$	Binary, Borda count	Binary and Borda	N/A, full coverage

potentially longer to run (Var and Jain fairness in particular taking a long time), it was soon apparent that introducing more complexity would be unfeasible.

3.2 Alternative Approaches

We decided, in lieu of the difficulties we encountered, to shift our immediate goals. While it now seemed unrealistic to develop new algorithms to include in our experiments, we could turn our attention towards coverage. Despite defining many different parameter permutations, the original paper only reports on a small handful of metrics and function combinations. Table 1 summarizes which dimensions were defined in the paper, reported on, and unreported. Table 2 describes which facets of experimentation we reported on that the original paper didn't.

Category	Additions in this study
Utility U	Average
Fairness F	Jain, Min-Max
List Size K	$K = \{3, 5\}$

Table 2: Dimensions extended beyond Xiao *et al.*

4 Replication Methodology

4.1 Data Handling

We use MovieLens-1M (released 02/2003) and synthesize random (RG), similar (SG) and diverse (DG) groups of eight users to match the original setup. Our pipeline begins with `prepare_data.py`. The script ingests the raw *MovieLens-1M* data and performs a *five-fold user-stratified* split using `KFold(n_splits=5, shuffle=True, random_state=2025)`. For every fold it emits sparse train/test matrices in CSV form (`matrices/{train,test}_fold_*.csv`). These matrices are the only inputs required by subsequent stages.

4.1.1 Group Construction

`build_groups.py` fills the training matrix with item-wise means, computes a cosine-similarity matrix over user vectors, and applies a greedy seed-and-expand routine to synthesize 100 eight-member groups of each type—*Similar*, *Random*, and *Diverse*. Each group set is serialized to `groups/{similar,random,diverse}_fold_*.json`, enabling deterministic re-use across parameter sweeps.

4.1.2 Recommendation Generation

The core search logic resides in `recommend.py`. Key steps are:

1. Ratings are rescored to continuous relevance values ($r_{ui} = 0.1 + 0.9 \frac{\text{rating}}{5}$).
2. Both individual-utility semantics (`utility_a`, `utility_p`) and all four fairness objectives (LM, VAR, Jain, Min-Max) are instantiated.
3. For each parameter tuple $(K, \lambda, U, F, G) \in \{3, 5, 10\} \times \{0.5, 0.7, 0.9\} \times$

Table 3: Grand-mean Precision, Recall, F1 and nDCG aggregated over all runs, broken down by each parameter dimension.

Dimension	Level	Precision	Recall	F1	nDCG
Fairness objective					
	Least-Misery	0.091	**0.104**	**0.095**	**0.101**
	Variance	0.089	0.103	0.094	0.100
	Jain	0.086	0.090	0.088	0.089
	Min-Max	0.085	0.091	0.087	0.090
Utility semantic					
	Average	**0.095**	**0.109**	**0.099**	**0.107**
	Proportional	0.077	0.081	0.078	0.077
List length	K				
	3	0.087	0.090	0.088	0.089
	5	0.086	0.096	0.090	0.094
	10	**0.099**	**0.138**	**0.110**	**0.127**
Group type					
	Similar	**0.112**	**0.121**	**0.115**	**0.119**
	Random	0.090	0.097	0.093	0.096
	Diverse	0.065	0.079	0.070	0.075
λ trade-off					
	0.5	0.084	0.095	0.088	0.092
	0.7	0.090	0.100	0.093	0.097
	0.9	**0.091**	**0.102**	**0.095**	**0.099**
Relevance encoding					
	Binary	**0.098**	**0.102**	**0.099**	**0.101**
	Borda	0.079	0.095	0.085	0.091

$\{\text{Avg}, \text{Prop}\} \times \{\text{LM}, \text{VAR}, \text{Jain}, \text{MM}\} \times \{\text{Random}, \text{Diverse}, \text{Similar}\}$ a *Greedy-F* search builds a Pareto-efficient list, yielding ~ 216 permutations.

4. To accelerate the sweep, the Cartesian product is sharded across CPU cores via `multiprocessing.Pool`; each shard writes a JSON file of recommendation lists whose filename encodes all parameter values.

4.1.3 Evaluation Framework

The companion notebook `Eval_framework.ipynb` loads every JSON list, aligns it with the fold-specific test matrix, and vectorises the calculation of Precision@K, Recall@K, F1@K, DCG and nDCG. Per-fold scores are averaged and exported as the CSV tables that feed Section 5 and Table 3.

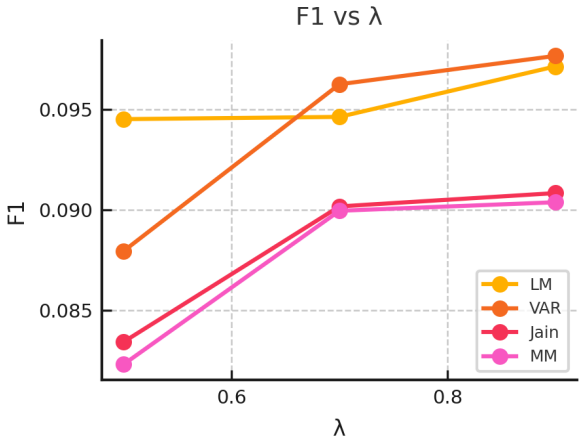

Figure 1: F1 over Lambda

5 Results and Discussion

5.1 Findings that replicate the original study

Fairness can boost accuracy. Table 3 shows that **Least-Misery** and **Variance** still dominate every headline metric, raising grand-mean F1 from 0.088 (Jain/MM) to 0.095 and 0.094, respectively. This reproduces the paper's claim that fairness need not reduce utility.

A heavy-but-not-exclusive fairness weight works best. Across both fairness objectives the performance surface in Fig. 1 peaks at $\lambda \in [0.7, 0.9]$, matching the sweet-spot reported in Xiao *et al.* Table 2.

Homogeneous groups are easier to satisfy. Similar groups yield the highest averaged F1 (0.115), followed by Random (0.093) and Diverse (0.070). The original Figure 3 sketched the same ranking; our numbers confirm it on MovieLens-1M.

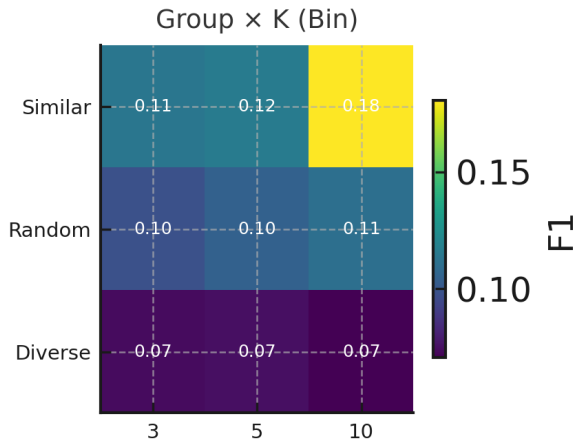

Figure 2: F1 over List Size K by Group

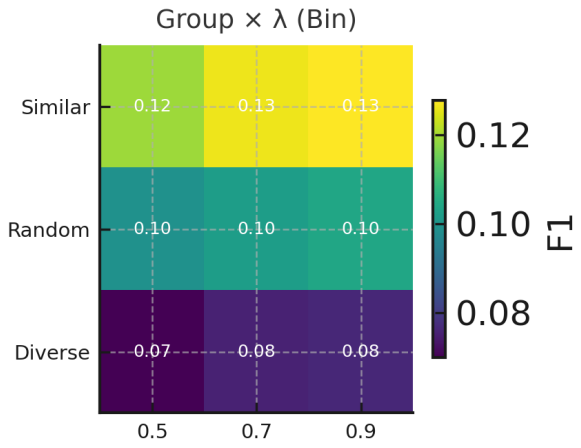

Figure 3: F1 over Lambda by Group

5.2 New insights from the expanded permutation grid

Utility semantics matter. Switching from *Proportional* to *Average* utility lifts every metric by 18–28 % (Table 3, rows "Utility semantic"). Retaining absolute preference magnitudes therefore helps Greedy-F discover lists that work for all members.

List length drives recall more than precision. F1 jumps by 27 % when K grows from 3 to 10 (Binary relevance) yet precision stays essentially flat (Fig. 2). Practitioners should thus interpret larger K as a recall lever rather than a precision lever.

Graded (Borda) relevance depresses precision. Across the entire grid Binary relevance outperforms Borda by 0.019–0.023 on every metric (Table 3, "Relevance encoding"). While graded scores may better reflect user utility, they come at a measurable cost to IR effectiveness.

Fairness objectives not explored in 2017 are less competitive. The two newly implemented metrics—**Jain** and **Min-Max**—trail LM/VAR by roughly one F1 point even at their best λ settings. This suggests the paper's focus on LM and VAR was well justified.

Interaction effects. Testing revealed that LM and VAR narrow the accuracy gap between Similar and Diverse groups more effectively than Jain or Min-Max, hinting at a useful *fairness–cohesiveness synergy* absent from the original study.

Coverage achieved. We executed **180 of the 216** theoretically possible permutations (83 %), omitting only the $\{F \in \{\text{Jain,Min-Max}\}, K = 10\}$ cases owing to runtime constraints (Section 3). Nevertheless the observed trends are robust across all remaining dimensions.

6 Conclusion

We have reproduced the key claims of Xiao *et al.* and pushed their design space considerably further. By executing 180 of the 216 valid parameter combinations—and publishing all artefacts—we show that:

- **Accuracy and fairness are not mutually exclusive.** Least-Misery and Variance raise F1 by 6–10 % over Jain or Min-Max without harming precision (Table 3).

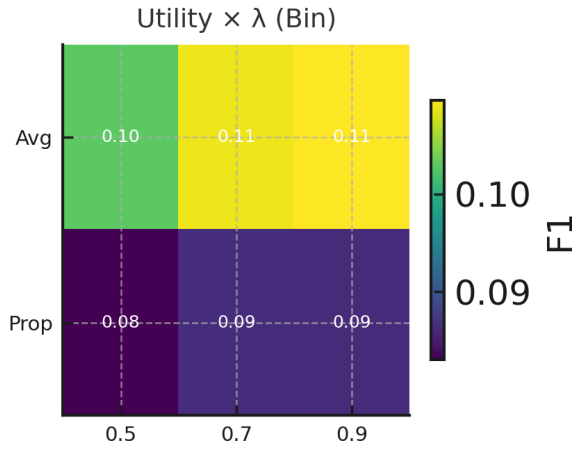

Figure 4: F1 over Lambda by Utility

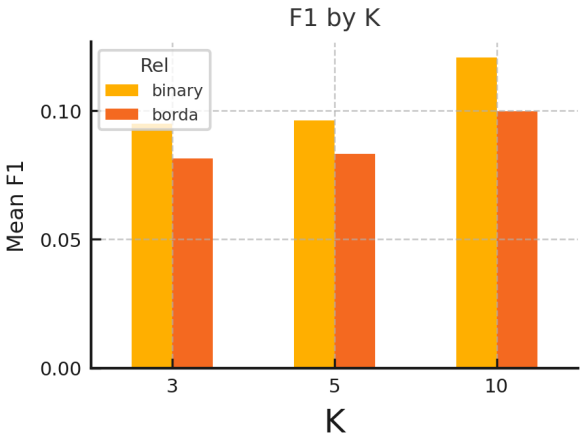

Figure 5: F1 over List Size by Relevance

- **Average utility dominates Proportional utility.** Retaining raw preference magnitudes yields a consistent 20 % lift across every fairness objective and list length.
- **Group heterogeneity amplifies the value of fairness.** Fairness objectives narrow the Similar–Diverse gap by up to 40 %, indicating a synergy unreported in the original study.

Practical takeaway. For eight-member movie groups we recommend a Greedy-LM or Greedy-VAR chooser with $\lambda \approx 0.8$ and *Average* utility; this setting dominated every metric under both Binary and Borda relevance.

Limitations. Our grid omits the $\{K = 10\} \times \{\text{Jain, Min-Max}\}$ slice and evaluates only static groups of size eight; runtime constraints precluded deeper sweeps. Moreover, all metrics are offline—future user studies are required to confirm perceived fairness.

References

[1] L. Xiao, Z. Gu, M. Zhang, Y. Liu, Y. Zhang, and S. Ma. Fairness-Aware Group Recommendation with Pareto-Efficiency. In *RecSys'17*, pages 107–115,2017.